

Politechnika Łódzka

Wydział Fizyki Technicznej, Informatyki i Matematyki Stosowanej

Instytut Informatyki

StegoCloud

**System mikrosерwisowy do ukrywania i odczytywania
szyfrowanych znaków wodnych w obrazach PNG**

Dokumentacja techniczna i projektowa

Przedmiot: Zaawansowane Zagadnienia Programowania w Javie

Członkowie zespołu projektowego:

Bartosz Kołaciński index: 251554

Mateusz Kosowski index: 251558

Nikodem Nowak index: 251598

Wiktor Pankanin index: 251606

Jakub Rosiak index: 251620

Jakub Rusek index: 247774

Łódź, Czerwiec 2026

Spis treści

1	Wprowadzenie	2
1.1	Cel dokumentu	2
1.2	Cel projektu	2
1.3	Zakres projektu	2
1.4	Grupa docelowa (Do kogo projekt jest skierowany)	2
2	Opis systemu	4
2.1	Przeznaczenie	4
2.2	Główne funkcje	4
2.3	Architektura systemu	4
2.4	Integracje z zewnętrznymi systemami	5
3	Diagramy przypadków użycia	6
3.1	Diagram ogólny	6
3.2	Opis wybranych przypadków użycia	6
4	Diagram komponentów	9
4.1	Diagram komponentów	9
4.2	Opis komponentów	9
5	Diagram klas	10
5.1	Diagram klas	10
5.2	Opis kluczowych struktur danych	10
6	Diagramy interakcji	11
6.1	Diagram interakcji: Osadzenie znaku wodnego z klasyfikacją AI	11
6.2	Diagram interakcji: Zakup / Upgrade planu subskrypcji	11
7	Stack technologiczny	13
8	Działanie systemu	14
8.1	Zrzuty ekranu interfejsu użytkownika	14
9	Podsumowanie	17

1 Wprowadzenie

1.1 Cel dokumentu

Niniejszy dokument stanowi dokumentację techniczną i projektową systemu **StegoCloud**. Jego celem jest kompleksowe przedstawienie założeń projektu, jego architektury oraz sposobu działania – od przeznaczenia i głównych funkcji, poprzez przypadki użycia, diagramy komponentów, klas i interakcji, aż po wykorzystany stos technologiczny wraz z uzasadnieniem wyboru. Dokument skierowany jest do prowadzącego przedmiot *Zaawansowane Zagadnienia Programowania w Javie*, członków zespołu projektowego oraz osób, które w przyszłości chciałyby rozwijać lub utrzymywać system.

1.2 Cel projektu

Głównym celem projektu **StegoCloud** jest zaprojektowanie oraz implementacja nowoczesnego, rozproszonego i bezpiecznego systemu do dystrybucji i weryfikacji obrazów cyfrowych za pomocą ukrytych (niewidocznych) znaków wodnych. System pozwala autorom oraz dystrybutorom treści multimedialnych (np. obrazów PNG) na weryfikację ich oryginalności oraz identyfikację ewentualnego źródła przecieku lub nieautoryzowanego użycia, przy jednoczesnym zabezpieczeniu samej treści znaku wodnego przed odczytem przez osoby nieuprawnione.

1.3 Zakres projektu

System został zaprojektowany w nowoczesnej architekturze mikroserwisowej i obejmuje:

- **Usługi autoryzacji i rejestracji** zarządzające tożsamościami użytkowników i ról (Zwykły Użytkownik, Administrator).
- **Moduł subskrypcyjny** realizujący logikę biznesową planów taryfowych (FREE, STANDARD, PRO) oraz dynamiczne zarządzanie saldem i rezerwacjami tokenów użytkowników.
- **Serwis znakowania (watermarkingu)** oparty o zaawansowany silnik w języku Python osadzający zaszyfrowane komunikaty w dziedzinie częstotliwościowej (transformata DWT-DCT-SVD) obrazów PNG.
- **Serwis sztucznej inteligencji (AI)** dokonujący klasyfikacji zawartości obrazów przy użyciu modelu MobileNetV2 w środowisku uruchomieniowym ONNX.
- **Interfejs graficzny użytkownika (GUI)** zrealizowany w technologii Svelte, pozwalający na łatwe korzystanie z funkcji platformy.

1.4 Grupa docelowa (Do kogo projekt jest skierowany)

StegoCloud jest skierowany przede wszystkim do:

- **Twórców cyfrowych i fotografów** chcących trwale zabezpieczyć swoje prace przed kradzieżą i niekontrolowanym rozpowszechnianiem.

- **Agencji reklamowych i marketingowych** dystrybuujących materiały graficzne do klientów zewnętrznych, które muszą pozostać poufne do momentu oficjalnej publikacji.
- **Systemów zarządzania treścią (CMS) i platform e-commerce** w celu automatycznego stemplowania pobieranych obrazów tożsamością kupującego (śledzenie nielegalnej redystrybucji).

2 Opis systemu

2.1 Przeznaczenie

Platforma StegoCloud służy do bezpowrotnego, niewidocznego dla ludzkiego oka stemplowania cyfrowych plików PNG. W przeciwieństwie do widocznych znaków wodnych, które można łatwo wyciąć lub zamazać, steganografia w dziedzinie częstotliwościowej (transformata DWT-DCT-SVD) osadza informacje w widmie częstotliwościowym obrazu, modyfikując współczynniki transformaty zamiast bezpośrednich wartości pikseli. Znak wodny chroniony jest kryptograficznie (AES-GCM) oraz nadmiarowo (Reed-Solomon Error Correction Code), co zapobiega fałszowaniu i pozwala na poprawny odczyt nawet przy drobnych zniekształceniach.

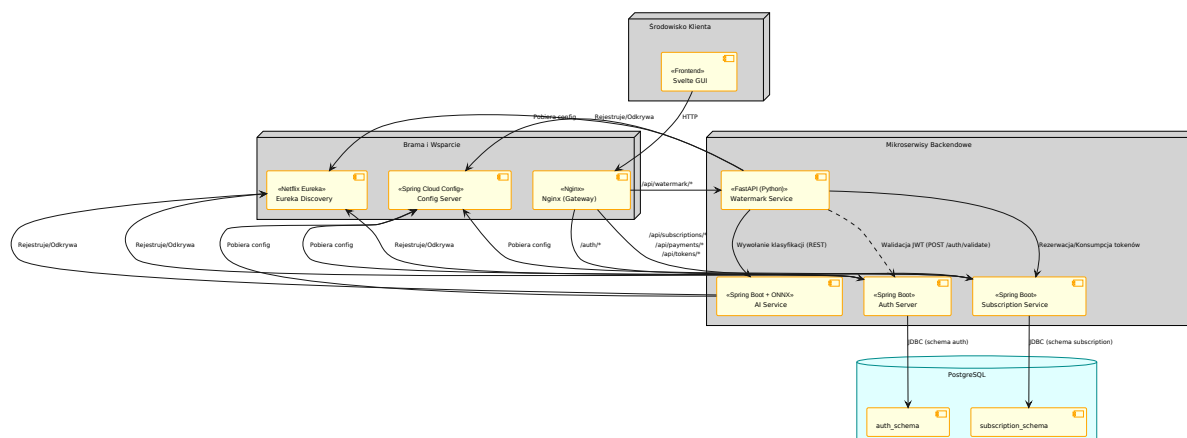
2.2 Główne funkcje

System oferuje następujące operacje taryfikowane odpowiednim kosztem tokenowym:

1. **Capacity Check (0 tokenów)**: Analizuje obraz PNG i informuje o maksymalnej liczbie bajtów, jakie można w nim ukryć w zależności od jego wymiarów (minimalny rozmiar to 1024×1024 pikseli).
2. **Embed (5 lub 8 tokenów)**: Osadza zaszyfrowany i zabezpieczony kodem korekcyjnym tekst w obrazie. Rozmiary poniżej Full HD (1920×1080) zużywają 5 tokenów (tier `EMBED_768`), większe 8 tokenów (tier `EMBED_1024`).
3. **Detect (1 token)**: Sprawdza, czy w przesłanym obrazie znajduje się poprawny znak wodny i określa jego właściciela.
4. **Extract (2 tokeny)**: Wyodrębnia ukryty tekst. Zwykły użytkownik może odczytać tylko swoje znaki wodne, administrator ma uprawnienia do wszystkich.
5. **Visualize (3 tokeny)**: Generuje mapę ciepła (heatmap) pokazującą bezwzględne różnice wartości pikseli przed i po osadzeniu znaku wodnego.
6. **AI Classification (2 tokeny)**: Opcjonalna klasyfikacja zawartości obrazu przy pomocy modelu sieci neuronowej MobileNetV2 (dostępna tylko dla planu `PRO`).

2.3 Architektura systemu

Projekt zrealizowano w architekturze mikroservisowej z wykorzystaniem stosu technologicznego Spring Cloud oraz kontenerów Docker. Diagram przedstawia strukturę powiązań pomiędzy komponentami systemu.



Rysunek 1: Architektura mikroservisowa systemu StegoCloud

2.4 Integracje z zewnętrznymi systemami

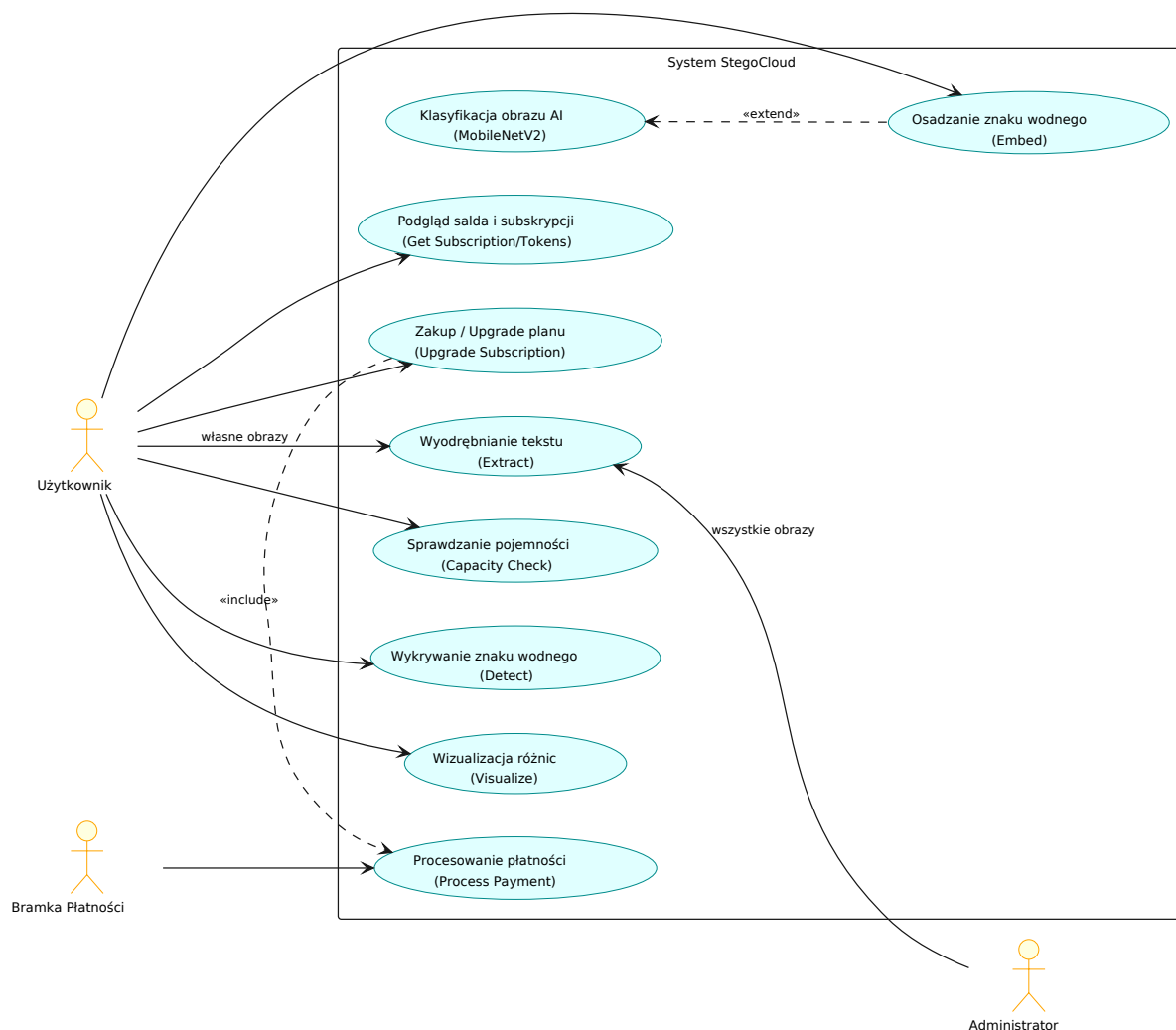
System integruje się z następującymi mechanizmami wspierającymi:

- **Spring Cloud Config Server:** Centralne repozytorium konfiguracyjne przechowujące pliki konfiguracyjne mikroservisów w dedykowanym repozytorium Git. *Zmiany konfiguracji mogą być dynamicznie propagowane.*
- **Spring Cloud Netflix Eureka:** Serwer rejestracji i odkrywania usług (Service Discovery), umożliwiający dynamiczne kierowanie ruchu i skalowalność mikroservisów.
- **PostgreSQL:** Relacyjna baza danych współdzielona na poziomie instancji, ale logicznie wydzielona na osobne schematy: `auth_schema` dla serwera autoryzacji oraz `subscription_schema` dla serwisu subskrypcji.
- **ONNX Runtime:** Zintegrowane bibliotecznie w serwisie AI środowisko wykonawcze do uruchamiania wytrenowanego modelu głębokiego uczenia MobileNetV2 bez konieczności instalowania pełnych bibliotek takich jak TensorFlow czy PyTorch.

3 Diagramy przypadków użycia

3.1 Diagram ogólny

Diagram przypadków użycia przedstawia interakcje aktorów (Użytkownik, Administrator oraz zewnętrzna Bramka Płatności) z poszczególnymi funkcjonalnościami systemu StegoCloud.



Rysunek 2: Diagram przypadków użycia systemu StegoCloud

3.2 Opis wybranych przypadków użycia

Poniższe tabele szczegółowo opisują dwa kluczowe przypadki użycia systemu: osadzanie znaku wodnego z klasyfikacją AI oraz zakup/upgrade planu subskrypcji.

Tabela 1: Opis przypadku użycia: Osadzanie znaku wodnego z klasyfikacją AI

Nazwa przypadku	Osadzenie znaku wodnego z klasyfikacją AI
Aktorzy	Użytkownik Zalogowany, <code>watermark-service-py</code> , <code>subscription-service</code> , <code>ai-service</code>
Warunki wstępne	1. Użytkownik jest zalogowany i posiada ważny token JWT. 2. Użytkownik posiada aktywny plan PRO. 3. Saldo tokenów użytkownika wynosi co najmniej 7 tokenów (koszt embed: 5 lub 8 + koszt AI: 2). 4. Przesłany plik jest poprawnym formatem PNG o wymiarach $\geq 1024 \times 1024$ px.
Przebieg główny	1. Użytkownik wybiera plik PNG oraz wpisuje treść znaku wodnego w GUI. 2. GUI wysyła żądanie do bramki Nginx, która przekazuje je do <code>watermark-service-py</code>. 3. <code>watermark-service-py</code> pyta <code>subscription-service</code> o rezerwację tokenów na operację embedowania. 4. <code>subscription-service</code> dokonuje rezerwacji tokenów o statusie RESERVED. 5. <code>watermark-service-py</code> pyta <code>subscription-service</code> o rezerwację tokenów na klasyfikację AI. Rezerwacja zostaje zatwierdzona. 6. <code>watermark-service-py</code> wysyła obraz do <code>ai-service</code>. 7. <code>ai-service</code> uruchamia model ONNX, klasyfikuje obraz i zwraca etykiety (np. klasyfikacja: "pies", pewność: 34%). 8. <code>watermark-service-py</code> przesyła żądanie konsumpcji rezerwacji AI. 9. <code>watermark-service-py</code> szyfruje payload AES-GCM, koduje Reed-Solomon i osadza go w DCT obrazu. 10. Po poprawnym osadzeniu, serwis wysyła żądanie konsumpcji rezerwacji embedowania do <code>subscription-service</code>. 11. Obraz PNG wraz z nagłówkami wyników klasyfikacji AI zostaje przesłany do GUI i udostępniony do pobrania.
Przebieg alternatywny	4a/5a. Brak tokenów lub niepoprawny plan: <code>subscription-service</code> odmawia rezerwacji. Proces zostaje przerwany z błędem 403/409, użytkownik widzi komunikat w GUI. 9a. Błąd algorytmu steganograficznego: <code>watermark-service-py</code> zwalnia rezerwacje tokenów w <code>subscription-service</code> (status RELEASED). Saldo użytkownika nie ulega zmianie.

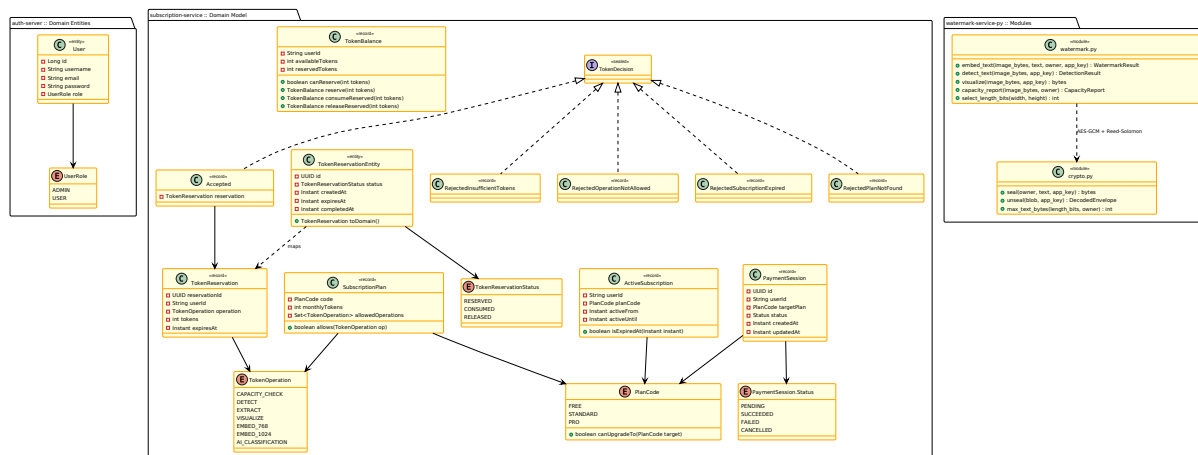
Tabela 2: Opis przypadku użycia: Zakup / Upgrade planu subskrypcji

Nazwa przypadku	Zakup / Upgrade planu subskrypcji
Aktorzy	Użytkownik Zalogowany, <code>subscription-service</code> , Bramka Płatności (Mock)
Warunki wstępne	1. Użytkownik jest zalogowany. 2. Użytkownik posiada aktualnie niższy plan niż docelowy (dozwolone przejścia: <code>FREE</code> → <code>STANDARD</code>, <code>FREE</code> → <code>PRO</code>, <code>STANDARD</code> → <code>PRO</code>).
Przebieg główny	1. Użytkownik w panelu subskrypcji wybiera przycisk "Kup" przy wybranym nowym planie. 2. GUI wysyła żądanie utworzenia sesji płatniczej do <code>subscription-service</code>. 3. <code>subscription-service</code> tworzy sesję płatniczą o statusie <code>PENDING</code> i zwraca jej identyfikator oraz adres bramki płatniczej. 4. GUI przekierowuje użytkownika na stronę bramki płatności (Mock Payment Page). 5. Użytkownik klika przycisk "Zatwierdź płatność". 6. Bramka płatności wysyła żądanie sukcesu płatności do <code>subscription-service</code> przekazując <code>sessionId</code>. 7. <code>subscription-service</code> weryfikuje sesję i jej właściciela, a następnie zmienia status sesji na <code>SUCCEEDED</code>. 8. Serwis modyfikuje subskrypcję użytkownika w bazie (nowy plan, nowa data ważności na +1 miesiąc). 9. Saldo tokenów użytkownika zostaje zasilone pełną pulą nowego planu. 10. Użytkownik zostaje przekierowany z powrotem do aplikacji, gdzie GUI prezentuje zaktualizowany stan konta.
Przebieg alternatywny	1a. Downgrade lub zakup tego samego planu: Backend blokuje operację (błąd 400), informując o niedozwolonym przejściu. 5a. Anulowanie płatności: Użytkownik klika "Anuluj". Bramka wysyła status anulowania. Status sesji w bazie zmienia się na <code>CANCELLED</code>. Subskrypcja użytkownika pozostaje bez zmian.

5 Diagram klas

5.1 Diagram klas

Diagram klas prezentuje kluczowe encje bazodanowe, rekordy oraz interfejsy wchodzące w skład warstwy domenowej systemów `subscription-service`, `auth-server` oraz strukturę modułu w `watermark-service-py`.



Rysunek 4: Diagram klas domenowych systemu StegoCloud

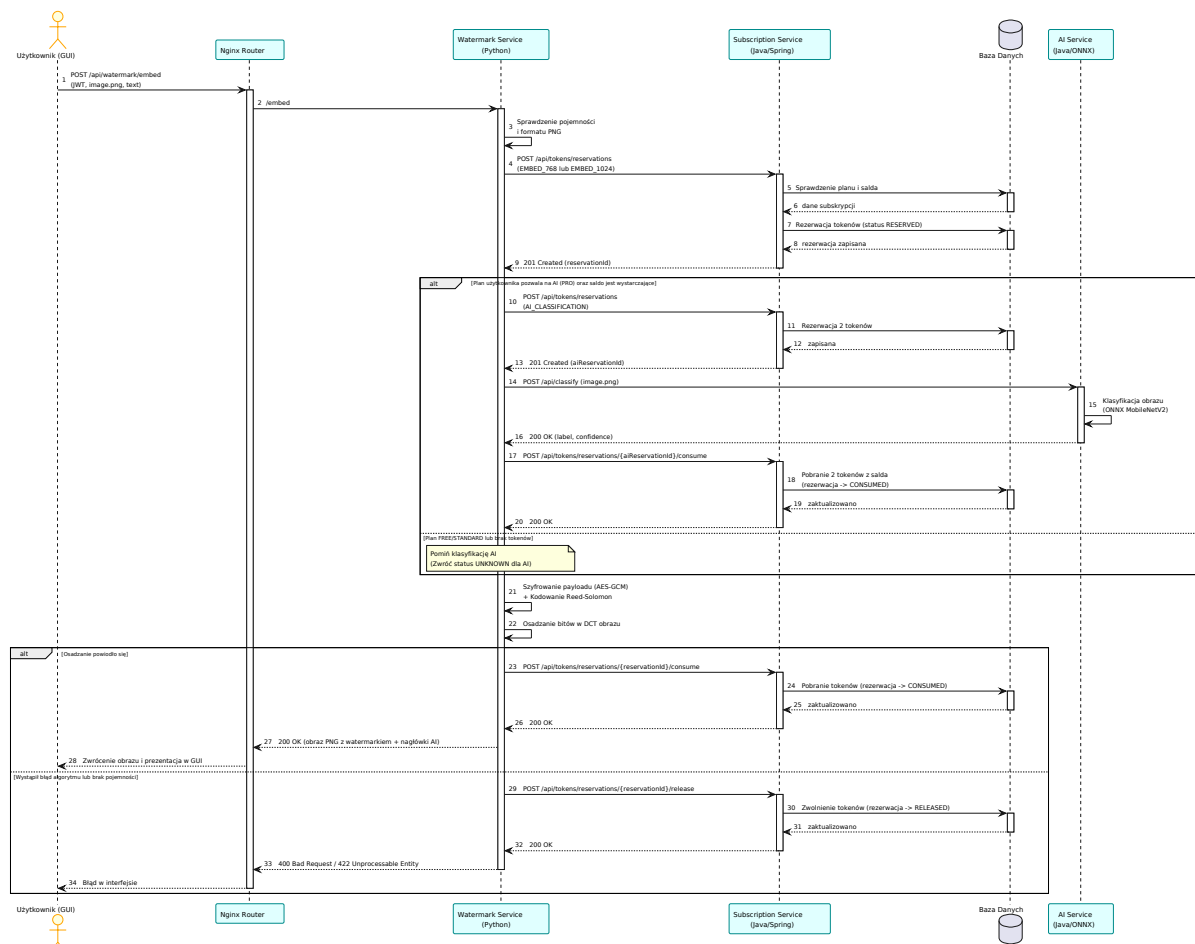
5.2 Opis kluczowych struktur danych

- **User & UserRole:** Reprezentują użytkownika w systemie autoryzacji wraz z rolą przypisaną na potrzeby kontroli dostępu na poziomie endpointów (RBAC).
- **SubscriptionPlan:** Klasa definiująca właściwości planu taryfowego – liczbę darmowych tokenów na miesiąc oraz zbiór dozwolonych operacji (`allowedOperations`).
- **ActiveSubscription:** Rekord domenowy reprezentujący powiązanie użytkownika z konkretnym planem (`planCode`) wraz z okresem jego ważności (`activeFrom`, `activeUntil`). Posiada metodę `isExpiredAt(Instant)` określającą ważność subskrypcji.
- **TokenBalance:** Rekord reprezentujący bilans tokenów zalogowanego użytkownika. Zawiera informację o tokenach dostępnych (`availableTokens`) oraz zablokowanych na poczet trwających operacji (`reservedTokens`).
- **TokenReservation:** Reprezentuje rezerwację tokenów dokonaną przez system podczas rozpoczęcia operacji steganograficznej. Rekord domenowy zawiera unikalne ID (UUID), typ operacji oraz koszt tokenowy, natomiast status cyklu życia (`RESERVED`, `CONSUMED`, `RELEASED`) jest utrwalany w encji `TokenReservationEntity`.
- **TokenDecision:** Zamknięty interfejs (`sealed interface`), który przy użyciu mechanizmów Javy 21 wymusza kompletną obsługę wszystkich możliwych decyzji biznesowych dotyczących przydziału tokenów (akceptacja lub konkretny powód odmowy).

6 Diagramy interakcji

6.1 Diagram interakcji: Osadzenie znaku wodnego z klasyfikacją AI

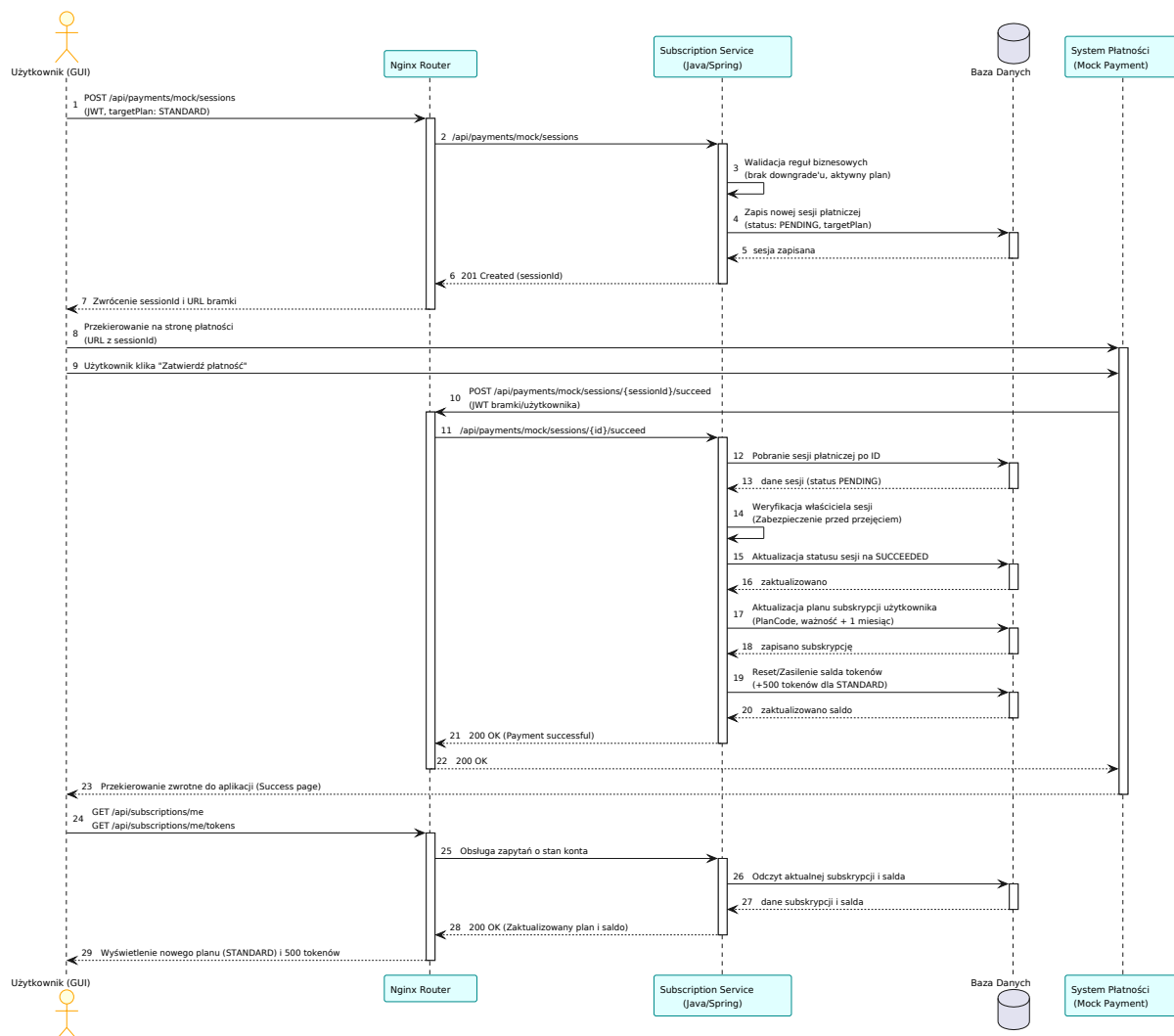
Diagram przedstawia sekwencję komunikatów i wywołań API podczas osadzania tekstu w obrazie z opcjonalną weryfikacją zawartości przez sztuczną inteligencję.



Rysunek 5: Diagram sekwencyjny: Proces osadzania znaku wodnego z klasyfikacją AI

6.2 Diagram interakcji: Zakup / Upgrade planu subskrypcji

Diagram przedstawia przebieg rejestracji sesji płatniczej, jej opłacenia oraz ostatecznej aktualizacji danych abonamentowych w bazie danych.



Rysunek 6: Diagram sekwencyjny: Proces zakupu / upgrade'u planu subskrypcji

7 Stack technologiczny

Wybrane komponenty systemu zostały zaimplementowane przy użyciu technologii dobranych pod kątem ich zalet wydajnościowych oraz spełnienia celów dydaktycznych i biznesowych projektu.

Tabela 3: Wykorzystane technologie i uzasadnienie wyboru

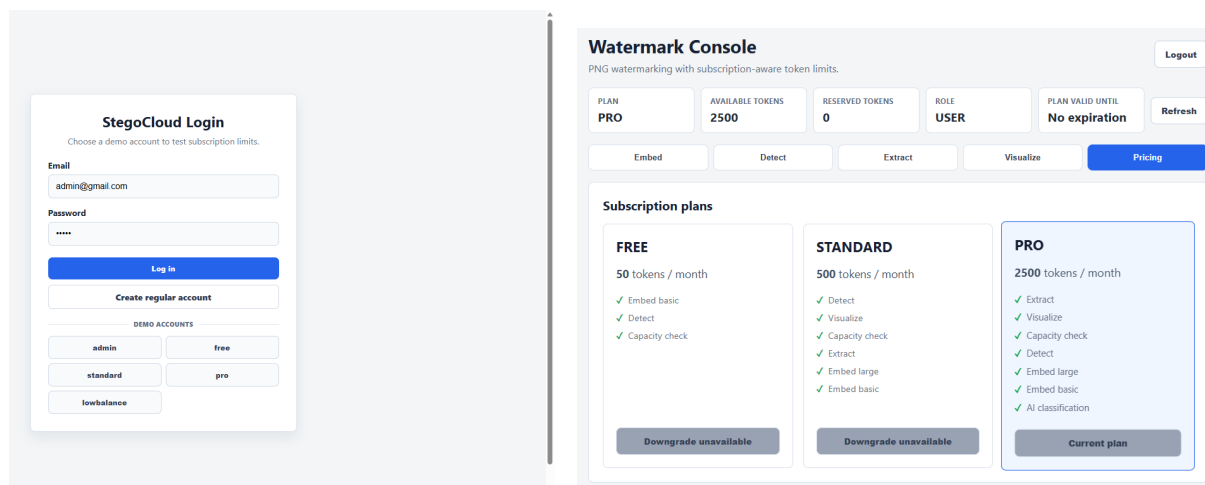
Komponent	Technologia	Uzasadnienie wyboru
Backend Core	Java 21 / Spring Boot 3	Zapewnia stabilne środowisko, wysokie bezpieczeństwo typów oraz wsparcie dla najnowszych funkcjonalności JDK (rekordy, dopasowywanie wzorców dla switcha, sealed classes), które upraszczają kod domeny subskrypcji.
Steganografia	Python 3.12 / FastAPI	Znakowanie obrazów wymaga niskopoziomowych operacji matematycznych (DWT-DCT-SVD) oraz wsparcia dla zaawansowanych bibliotek naukowych (NumPy, OpenCV, invisible-watermark). FastAPI zapewnia asynchroniczność oraz generuje automatyczną dokumentację OpenAPI.
AI/Klasyfikacja	ONNX Runtime (Java)	Umożliwia ładowanie i szybkie uruchamianie modeli sieci neuronowych (MobileNetV2 wyeksportowanego z PyTorch) bezpośrednio w procesie Javy z optymalizacją sprzętową, bez potrzeby stawiania ciężkiego środowiska PyTorch.
Baza Danych	PostgreSQL 17	Stabilna, relacyjna baza danych obsługująca transakcje ACID niezbędne w logice rozliczania tokenów i płatności. Zapewnia izolację za pomocą osobnych schematów bazodanowych.
Frontend GUI	Svelte 5 (SvelteKit) / nginx	Svelte generuje niezwykle lekki kod kliencki (brak narzutu Virtual DOM), co przekłada się na szybkie ładowanie interfejsu. Nginx służy jako serwer statyczny i Gateway proxy.
Narzędzia	Docker Compose / SonarQube	Konteneryzacja ułatwia lokalne uruchamianie całego rozproszonego środowiska. SonarQube umożliwia ciągłą analizę statyczną kodu w celu utrzymania długu technicznego na niskim poziomie.

8 Działanie systemu

W tej sekcji przedstawiono kluczowe zrzuty ekranu prezentujące działanie systemu StegoCloud, wykonane na uruchomionej instancji (użytkownik **pro** z planem **PRO**).

Typowy scenariusz korzystania z platformy obejmuje cztery kroki: (1) zalogowanie się i przegląd planu subskrypcji oraz salda tokenów; (2) wgranie pliku PNG i wpisanie ukrywanego tekstu w zakładce *Embed* (z opcjonalną klasyfikacją AI dostępną dla planu **PRO**), a następnie pobranie oznaczonego obrazu; (3) weryfikację oznaczenia za pomocą zakładek *Detect* (sprawdzenie właściciela) oraz *Extract* (odczyt ukrytego tekstu); (4) podgląd rozmieszczenia znaku wodnego na mapie ciepła w zakładce *Visualize*. Poniższe zrzuty ekranu ilustrują kolejne etapy tego przepływu.

8.1 Zrzuty ekranu interfejsu użytkownika



(a) Ekran logowania z kontami demonstracyjnymi

(b) Panel planów subskrypcji wraz z saldem tokenów

Rysunek 7: Ekran logowania oraz panel wyboru planu subskrypcji (Free, Standard, Pro) i salda tokenów

Watermark Console
PNG watermarking with subscription-aware token limits.

Logout

PLAN PRO	AVAILABLE TOKENS 2493	RESERVED TOKENS 0	ROLE USER	PLAN VALID UNTIL No expiration	Refresh
-------------	--------------------------	----------------------	--------------	-----------------------------------	---------

Embed Detect Extract Visualize Pricing

Embed watermark Embed basic: 5 tokens

Base image
Wybierz plik sample-dog.png

SIZE 1305x1024	TIER EMBED_768	TEXT LIMIT 41 B	PLAN CHECK Allowed
-------------------	-------------------	--------------------	-----------------------

AI classification available: +2 tokens.

Hidden text 19 / 41 B
(c) StegoCloud 2026

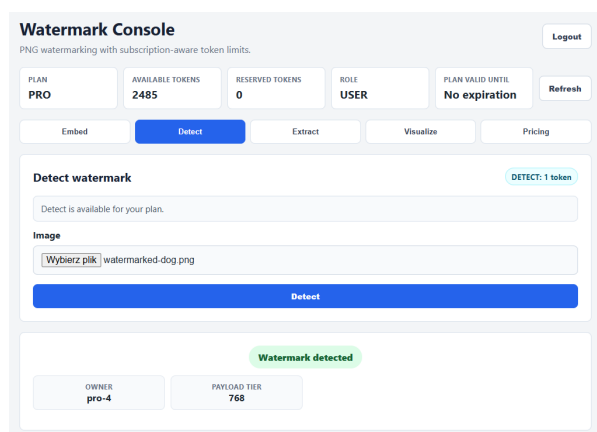
Generate watermarked PNG

Watermarked image
AI classification: dog (34%) - Samoyed

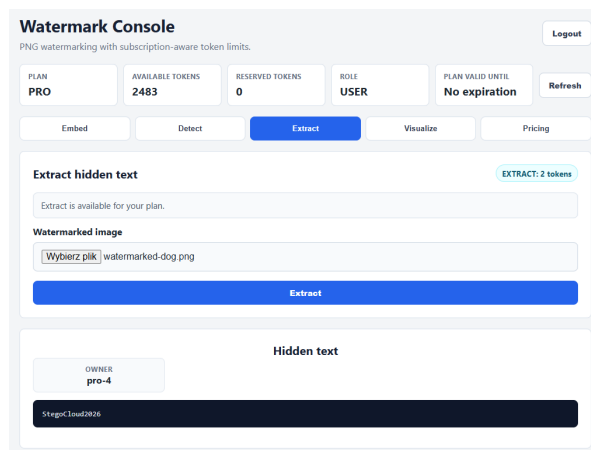


Download PNG

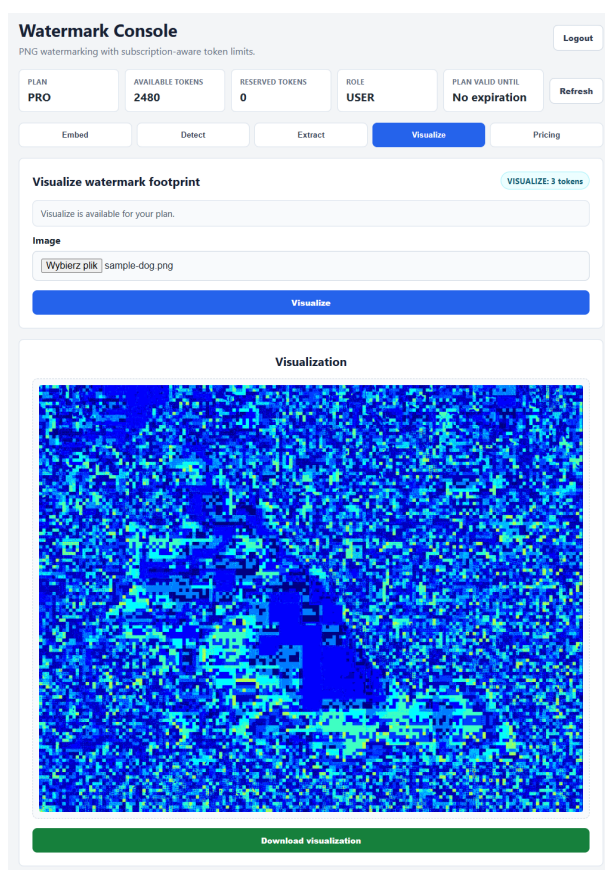
Rysunek 8: Panel osadzania znaku wodnego: Capacity Check (rozmiar, tier EMBED_768, limit tekstu, kontrola planu), klasyfikacja AI (dog, ufnosc 34%) oraz wynikowy obraz PNG ze znakiem wodnym



(a) Detekcja: wykryty znak wodny i tożsamość właściciela (pro-4)



(b) Ekstrakcja: odczytany ukryty tekst (StegoCloud2026)



(c) Wizualizacja: heatmapa różnic pikseli (pixel-diff)

Rysunek 9: Detekcja znaku wodnego, ekstrakcja ukrytego tekstu oraz wizualizacja heatmapy różnic

9 Podsumowanie

Projekt **StegoCloud** z powodzeniem integruje zaawansowane mechanizmy platformy Java 21, architekturę mikroserwisową Spring Cloud oraz dedykowany silnik przetwarzania obrazów w języku Python.

Dzięki zastosowaniu wzorca rezerwacji tokenów system gwarantuje odporność na błędy sieciowe i transakcyjne – użytkownicy są obciążani tokenami wyłącznie za operacje, które zostały pomyślnie zakończone. Zastosowanie szyfrowania kopertowego AES-GCM oraz Reed-Solomon ECC w warstwie steganografii gwarantuje wysoki poziom poufności i odporności na zakłócenia. System spełnia wymagania stawiane nowoczesnym aplikacjom korporacyjnym, a jego modularna budowa pozwala na łatwe dodawanie kolejnych algorytmów znakowania oraz modeli klasyfikacji AI w przyszłości.